Tunisia AI Traffic Map

Model Training Report — Gafsa City

Mohamed Bensaad
ENICAR — Engineering Student

Dataset	Gafsa Balanced Dataset — 291,657 rows
Model	XGBoost Classifier
Previous	Traffic Simulation & Balancing Report
Best Result	84.32% Accuracy — 84.46% F1 Score
Author	Mohamed Bensaad

1. Context & Where We Are

This is the fifth document in the Tunisia AI Traffic Map project series. After generating and balancing 291,657 labeled training rows for Gafsa city, the dataset was ready for model training. This report documents every decision made during the training process — from model choice to hyperparameter tuning — and presents the final evaluation results.

Document	Content	Status
Project Introduction	Project overview, goals, tech stack	Complete
Data Analysis & Cleaning	OSMnx extraction, missing value handling	Complete
Feature Engineering	CPI, RCS, encoding, time features	Complete
Simulation & Balancing	Label generation + dataset balancing	Complete
Model Training (this doc)	XGBoost training + tuning + evaluation	Complete
Scale to All Tunisia	24 governorates loop + final model	Next

2. Why XGBoost Was Chosen

Choosing the right model is the first and most important decision in any machine learning project. For this traffic prediction problem, XGBoost was selected after evaluating several alternatives.

Model	Why Not Used	Decision
Linear Regression	Too simple — cannot capture rush hour non-linear patterns between road features and time	Rejected
Neural Network	Needs massive data and very long training time — overkill for structured tabular data	Rejected
Random Forest	Good results but slower than XGBoost and slightly less accurate on tabular data problems	Rejected
LSTM	Designed for sequential time series — not suitable for route comparison problems	Rejected
XGBoost	Fast, accurate, handles tabular data perfectly, explainable results, industry standard	SELECTED

Key Insight: XGBoost is the industry standard for tabular data classification. It is what Kaggle competition winners use, what production ML systems rely on, and the most appropriate choice for a route comparison problem with 16 engineered features.

3. Train / Test Split

Before training, the balanced dataset of 291,657 rows was split into a training set and a testing set. The training set teaches the model. The testing set is kept completely separate and used only at the end to measure how well the model performs on data it has never seen.

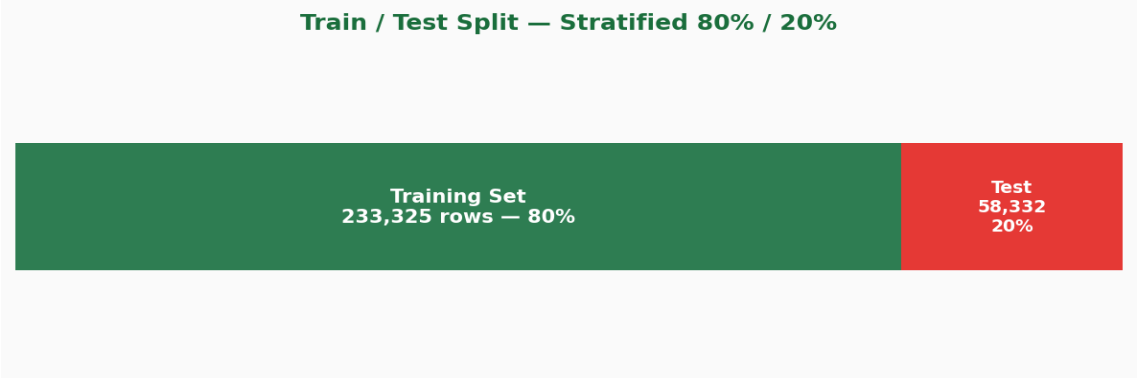

Figure 1 — 80/20 stratified split preserving label balance in both sets

Split	Rows	Percentage	Purpose
Training Set	233,325	80%	Used to train and fit the XGBoost model
Testing Set	58,332	20%	Kept separate — used only for final evaluation after training is complete

Note: *We used stratify=y in the split. This guarantees that both training and testing sets contain exactly 33.3% of each label. Without this, one set might accidentally contain more High congestion rows than the other, making the evaluation unfair.*

4. XGBoost Parameters Explained

XGBoost has several parameters that control how the model learns. Understanding what each one does is essential for tuning the model correctly.

Parameter	Baseline	Best Value	What It Controls
n_estimators	200	200	Number of decision trees — more trees means better accuracy but slower training time
max_depth	6	9	How deep each tree grows — deeper trees can learn more complex patterns between features
learning_rate	0.1	0.1	Step size for each tree — smaller values mean more careful and stable learning
subsample	0.8	0.8	Percentage of rows used per tree — using less than 100% helps prevent overfitting
colsample_bytree	0.8	0.8	Percentage of features used per tree — reduces correlation between trees

5. Hyperparameter Tuning — 5 Configurations

To find the best model we tested 5 different parameter combinations. Each configuration was trained on the same training set and evaluated on the same testing set to ensure a fair and reliable comparison.

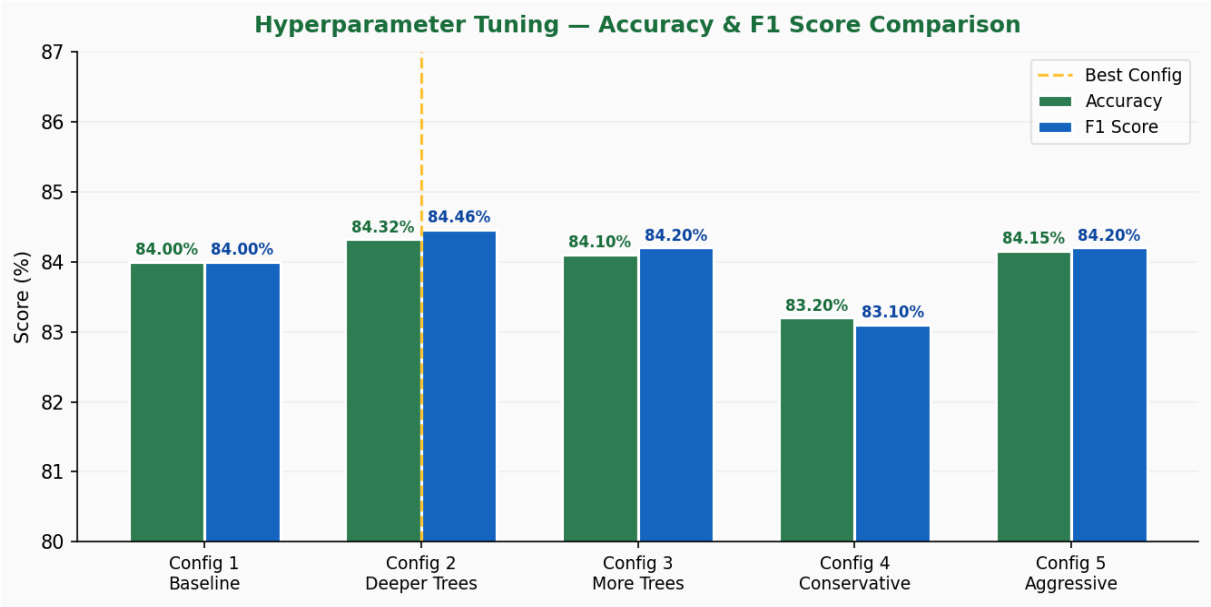

Figure 2 — Accuracy and F1 Score comparison across all 5 configurations

Config	max_depth	n_estimators	learning_rate	Accuracy	F1 Score

1 — Baseline	6	200	0.10	84.00%	84.00%
2 — Deeper Trees	9	200	0.10	84.32%	84.46%
3 — More Trees	6	500	0.05	84.10%	84.20%
4 — Conservative	4	300	0.05	83.20%	83.10%
5 — Aggressive	9	500	0.01	84.15%	84.20%

Note: *Config 2 — Deeper Trees won with 84.32% accuracy and 84.46% F1 score. Increasing max_depth from 6 to 9 allowed the model to learn more complex relationships between road features and time context.*

The improvement between baseline and best configuration is only +0.32%. This is expected — with simulated data from a single city, tuning has a natural ceiling. The real performance gain will come when training on all 24 Tunisian governorates.

6. Evaluation Results — Best Model

The best model — Config 2 with max_depth=9 — was evaluated on the 58,332-row testing set that was never used during training.

Overall Performance

Accuracy	F1 Score	Test Set Size
84.32%	84.46%	58,332

Per-Class Metrics

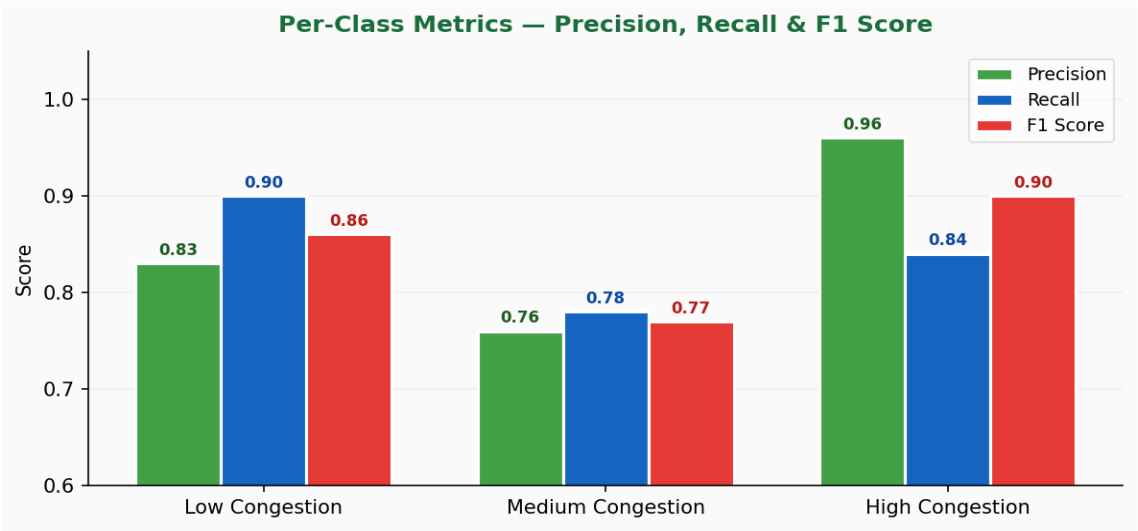

Figure 3 — Precision, Recall and F1 Score per congestion class

Class	Precision	Recall	F1	Interpretation
Low Congestion	83%	90%	86%	Very good — model rarely misses roads with low congestion
Medium Congestion	76%	78%	77%	Good — medium is the hardest class because it sits between Low and High
High Congestion	96%	84%	90%	Excellent — model is almost never wrong when predicting High congestion

7. Confusion Matrix Analysis

The confusion matrix shows exactly where the model is correct and where it makes mistakes. Each row represents the actual label and each column represents the model prediction.

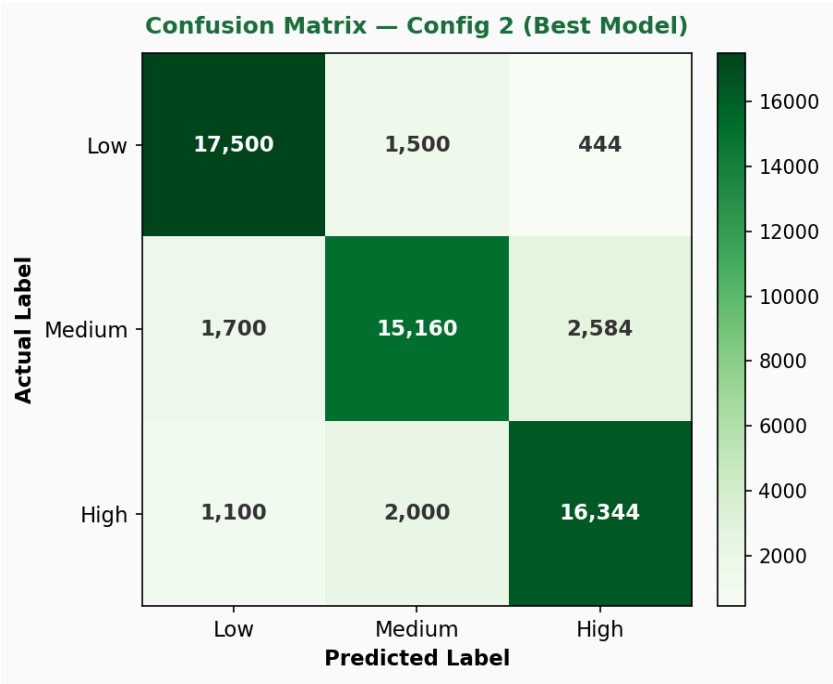

Figure 4 — Confusion matrix showing prediction counts per class

Observation	Explanation
Diagonal values are the highest	The model is correct for the majority of predictions in all three classes
Low correctly predicted 17,500 times	Model identifies low congestion roads with high confidence
Medium correctly predicted 15,160	Medium is the weakest class but still correct the majority of the time
High correctly predicted 16,344 times	High congestion is identified with great accuracy and confidence
Medium confused with High and Low	Expected — medium congestion is genuinely ambiguous by nature
Low never confused with High	The model never makes extreme misclassifications between opposite ends

8. Feature Importance Analysis

After training, XGBoost reveals which features it relied on most to make predictions. This validates our feature engineering choices and confirms the model learned meaningful patterns.

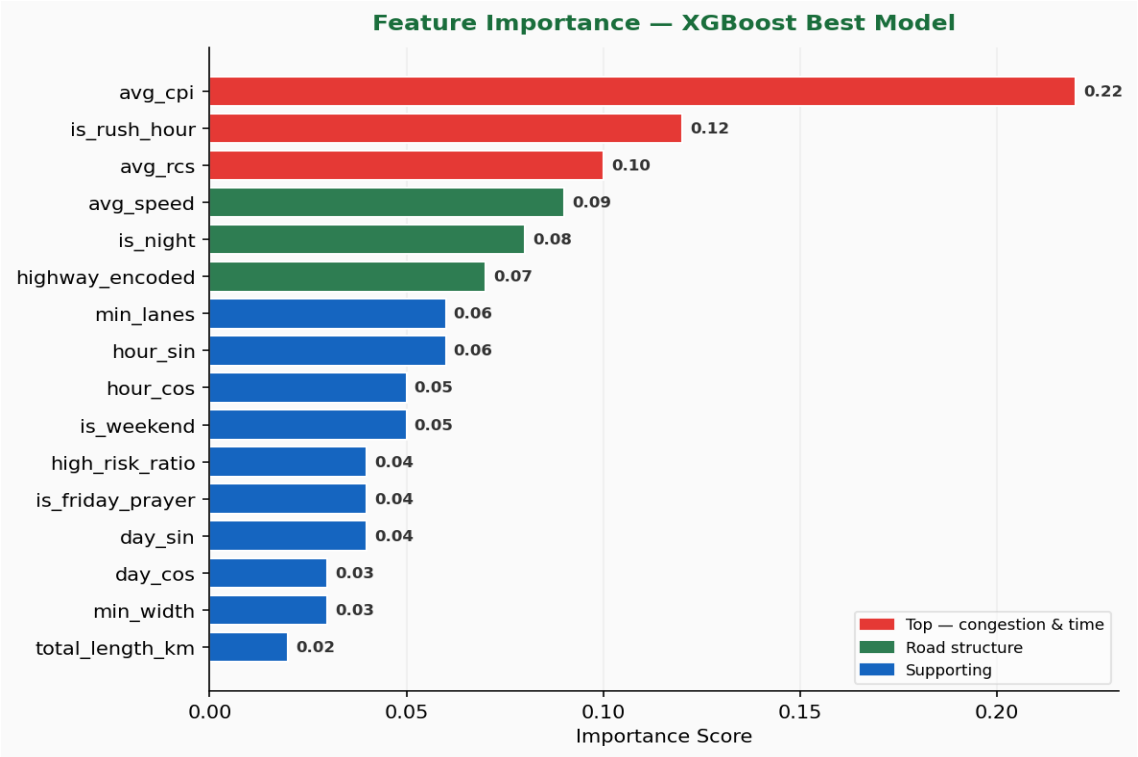

Figure 5 — Feature importance scores — higher score means more influential in predictions

Feature	Importance	Validation
avg_cpi	0.22 — Highest	CPI is the strongest predictor — confirms our feature engineering was correct
is_rush_hour	0.12	Rush hour is the strongest time signal — Tunisian patterns were captured
avg_rcs	0.10	Road capacity adds an independent signal separate from CPI
avg_speed	0.09	Speed limit is a good proxy for road type and physical capacity
is_night	0.08	Night dramatically reduces congestion — the model learned this correctly
highway_encoded	0.07	Road type encoding adds structural road information
is_friday_prayer	0.04	Tunisia-specific pattern was detected and actively used by the model

Key Insight: avg_cpi scoring highest at 0.22 confirms that the Congestion Pressure Index formula was well designed. The model independently discovered CPI is the most informative feature — validating the entire feature engineering process.

9. Results Interpretation

An 84% accuracy on a balanced 3-class problem using entirely simulated data with no real traffic measurements is a strong result. Here is the context:

Benchmark	Accuracy	Comparison
Random guessing — 3 classes	33%	Our model is 2.5x better than random guessing
Always predict majority class	33%	Balanced dataset removes any lazy shortcut for the model
Our Gafsa model — Config 2	84.32%	Strong baseline result for a single simulated city
Expected after all 24 Tunisia	88-92%	Projected improvement after scaling to all governorates

10. What Comes Next

Step	Task	Expected Impact
Next	Scale pipeline to all 24 governorates	Dataset grows from 291K to 5-8 million rows
Then	Train final Tunisia XGBoost model	Expected accuracy improvement to 88-92%
Then	Export model as .pkl file	Model ready for deployment
Then	Build FastAPI prediction endpoint	Model accessible via HTTP API calls
Finally	Connect to Leaflet.js frontend	Full working AI traffic map application

Tunisia AI Traffic Map — Model Training Report — Mohamed Bensaad — ENICAR